**МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ**
**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**

**Кафедра интеллектуальных и киберфизических систем**

# Лабораторная работа №4

## «Проектирование модели данных. Проектирование API и разработка архитектуры ИС»

Проект: «HammerLive» – информационная система для проведения онлайн-аукционов в реальном времени

Выполнили:
Студент 4 курса, 5 группы
Сиренко Иван Владиславович
Студент 4 курса, 5 группы
Гардееня Артем Сергеевич
Студентка 4 курса 5 группы
Голуб Александра Владимировна

Проверил:
преподаватель
Ломако Алексей Андреевич

Минск, 2026

# ВВЕДЕНИЕ

В работе спроектированы модель данных, программный интерфейс и архитектура информационной системы HammerLive на основе технического задания (Лабораторная работа №2) и UML-диаграмм (Лабораторная работа №3). Построены концептуальная и логическая ER-модели и сделан вывод о способе хранения данных, разработан прототип спецификации REST API для взаимодействия веб-клиента с подсистемой торгов, обосновано применение архитектурного паттерна CQRS.

# РАЗДЕЛ 1. ПРОЕКТИРОВАНИЕ МОДЕЛИ ДАННЫХ

## 1.1. Концептуальная модель данных

Концептуальная модель (рисунок 1) построена в нотации Чена и отражает основные сущности предметной области: пользователь, аукцион, лот, ставка, платёж, банковская карта и отзыв. Поскольку система построена на микросервисах, а каждый сервис владеет своими данными (см. диаграмму компонентов ЛР №3), сущности сгруппированы по сервисам-владельцам: аукцион и лот относятся к AuctionSession, ставка – к BidTracker, платёж и карта – к Payment, пользователь и отзыв – к вспомогательным сервисам Identity и Reputation. Роли пользователя (участник, продавец, организатор, аукционист) не выделены в отдельные сущности, так как один человек может выступать в нескольких ролях.

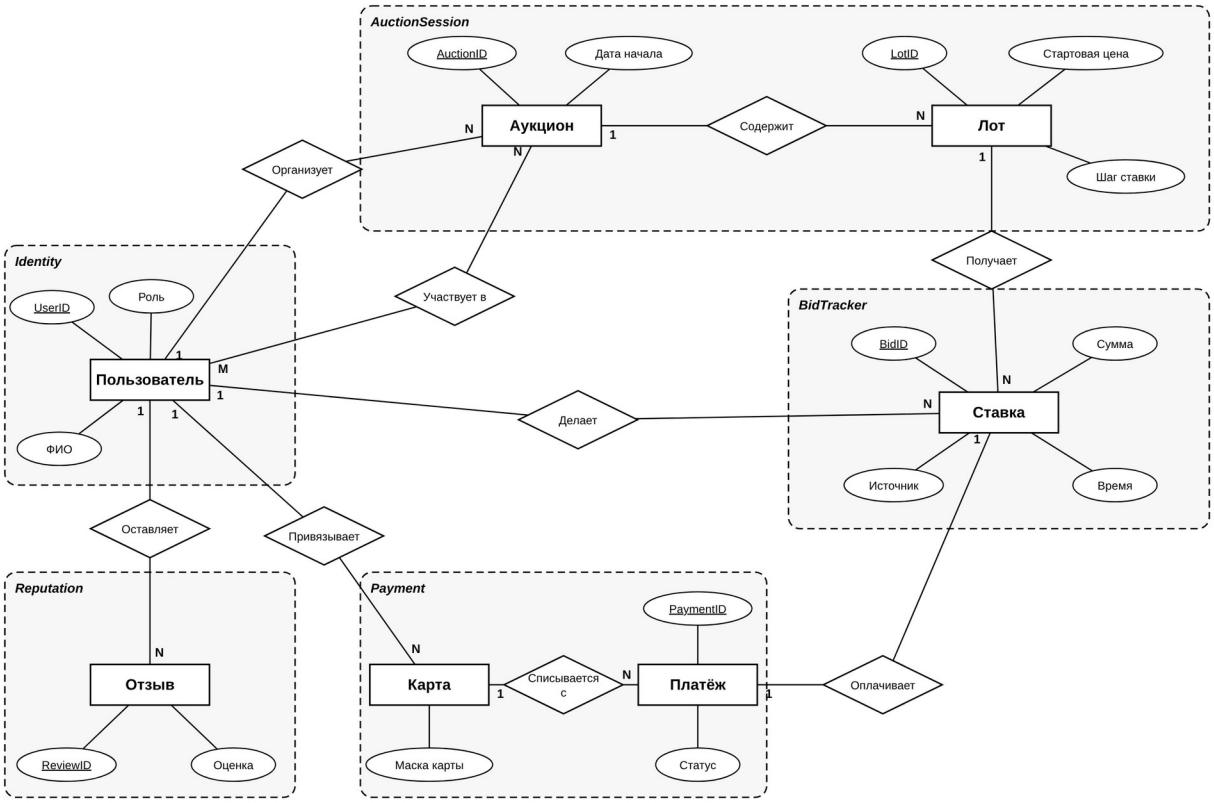



Рисунок 1 – Диаграмма «сущность-связь» (концептуальная модель)

Ключевые связи модели: пользователь организует аукционы (1:N) и участвует в аукционах (M:N); аукцион содержит лоты (1:N); лот получает ставки (1:N), каждую из которых делает один пользователь; ставка может поступить онлайн или из зала через аукциониста (атрибут «Источник»); выигрышная ставка оплачивается ровно одним платежом (1:1), который списывается с привязанной пользователем карты. Связи, пересекающие границы сервисов, на уровне хранения реализуются не внешними ключами, а ссылками по идентификатору и событиями.

### 1.2. Логическая модель данных

Логическая модель (рисунок 2) детализирует концептуальную до уровня таблиц, атрибутов, типов и ключей в нотации «вороньей лапки» и разбита по базам данных сервисов по принципу «отдельная база на сервис». Внешние ключи (FK) существуют только внутри одной базы; ссылки на данные другого сервиса хранятся как идентификаторы (ID) без внешнего ключа и показаны пунктиром.

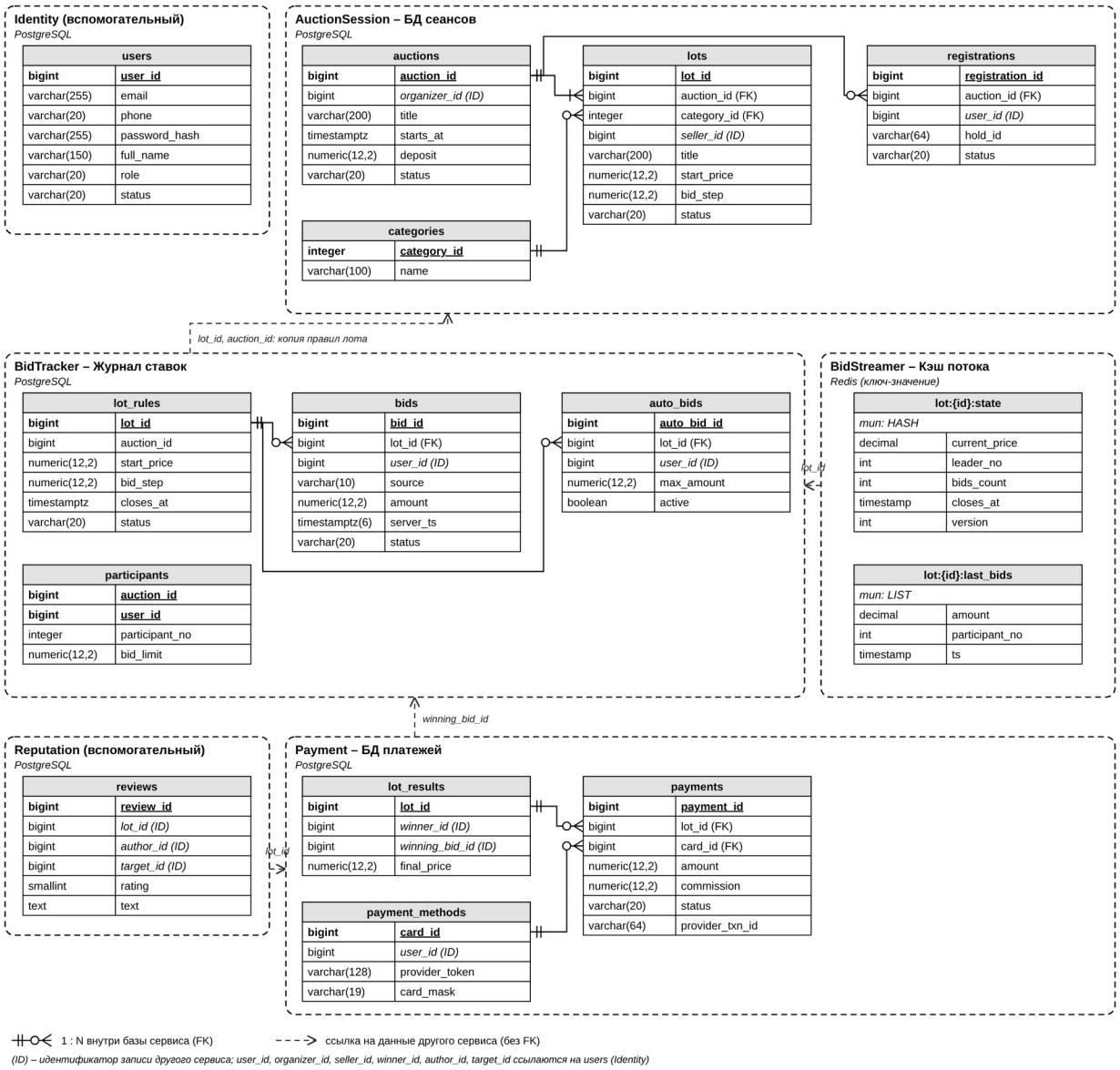



Рисунок 2 – Детализированная ER-диаграмма (логическая модель по сервисам)

База AuctionSession хранит аукционы, лоты, категории и регистрации участников – связь M:N «участвует в» разрешена через таблицу registrations. BidTracker не обращается к этой базе напрямую: необходимые для проверки ставок правила лота (стартовая цена, шаг, время закрытия) и список допущенных участников с их лимитами он хранит у себя в таблицах lot_rules и participants, получая их из событий сеанса. Это намеренное дублирование данных, которое делает BidTracker независимым от доступности AuctionSession во время торгов. Журнал bids хранит все ставки (поле source различает онлайн-ставки и ставки из зала), а bids.server_ts имеет микросекундную точность, так как по нему определяется порядок ставок.

BidStreamer хранит данные в Redis в виде структур «ключ-значение»: хеш lot:{id}:state с текущей ценой, лидером и таймером и список lot:{id}:last_bids с последними ставками. Payment хранит итоги лотов, полученные от BidTracker (lot_results), токены банковских карт и платежи; денежные суммы во всех базах имеют тип numeric(12,2). Пользователи (users) принадлежат сервису Identity, отзывы – сервису Reputation; остальные сервисы хранят только идентификаторы пользователей.

### 1.3. Выбор модели хранения данных

Данные внутри каждого сервиса сильно связаны и требуют транзакционной целостности: правила лота и ставки в BidTracker, платёж и итог лота в Payment должны изменяться атомарно. Поэтому для AuctionSession, BidTracker, Payment, Identity и Reputation выбрана **реляционная** модель (PostgreSQL) с нормализацией до третьей нормальной формы. Для BidStreamer, который только раздаёт текущее состояние торгов тысячам участников, выбрано хранилище **«ключ-значение»** (Redis): здесь важна скорость чтения, а данные всегда можно восстановить из журнала BidTracker.

Согласованность между базами обеспечивается не общими транзакциями, а асинхронными событиями через очереди: AuctionSession публикует события сеанса, BidTracker – принятые ставки и итоги лотов. Захватчики ставок (BidCapture, AuctioneerCapture) и видеосервисы собственных данных не хранят. Фото лотов и записи трансляций размещаются в объектном хранилище, а в базах хранятся только ссылки на них. Таблица bids, растущая быстрее остальных, секционируется по месяцам и индексируется по паре (lot_id, server_ts).

## РАЗДЕЛ 2. ПРОЕКТИРОВАНИЕ API

### 2.1. Выбор подхода к взаимодействию

Для проектирования выбрано взаимодействие веб-клиента с подсистемой торгов (сервисы BidCapture, BidTracker и BidStreamer через API Gateway) – оно определяет ключевую функцию системы. Сравнение подходов приведено в таблице 1.

Таблица 1 – Сравнение подходов к организации API

| Подход | Преимущества | Недостатки для HammerLive |
|---|---|---|
| REST (HTTP + JSON) | Простота, кэширование, поддержка браузерами и API Gateway, понятные коды ошибок | Не умеет сам отправлять данные клиенту – нужен дополнительный канал |
| GraphQL | Гибкие выборки данных одним запросом | Избыточен для небольшого набора операций, сложнее кэширование и контроль нагрузки |
| gRPC | Высокая скорость, строгие контракты | Нет прямой поддержки в браузере, подходит для связи сервисов между собой |
| WebSocket | Двусторонняя связь, мгновенная доставка событий | Нет стандартной семантики запросов и ошибок, сложнее повторы и идемпотентность |

Выбрано сочетание: **REST** для команд (сделать ставку, настроить автоставку) и запросов (состояние лота, история), **WebSocket** – только для рассылки событий от сервера. Это даёт стандартные HTTP-коды ошибок и идемпотентность для команд и мгновенную доставку обновлений всем участникам. Ставка обрабатывается асинхронно: BidCapture только принимает её и помещает в очередь, поэтому команда возвращает 202 Accepted, а результат проверки приходит событием по WebSocket. Взаимодействие сервисов между собой через очереди в спецификацию не входит.

### 2.2. Описание запросов

Базовый адрес API – https://api.hammerlive.by/v1. Все запросы передаются по HTTPS и содержат заголовок Authorization: Bearer <JWT>. Команды POST дополнительно содержат заголовок Idempotency-Key (UUID), чтобы повтор запроса при обрыве связи не создавал вторую ставку. Перечень операций приведён в таблице 2.

Таблица 2 – Операции API подсистемы торгов

| Метод | Путь | Назначение | Ответ |
|---|---|---|---|
| POST | /auctions/{auctionId}/ registrations | Регистрация на аукцион с холдом депозита | 201 |
| GET | /lots/{lotId}/state | Текущее состояние лота (BidStreamer) | 200 |
| POST | /lots/{lotId}/bids | Сделать ставку (BidCapture) | 202 |
| GET | /lots/{lotId}/bids?limit=&before= | История ставок с постраничной выдачей (BidTracker) | 200 |
| PUT | /lots/{lotId}/auto-bid | Создать или изменить автоставку | 200 |
| DELETE | /lots/{lotId}/auto-bid | Отключить автоставку | 204 |
| WS | /ws/lots/{lotId} | Подписка на события bid.accepted, bid.rejected, lot.extended, lot.closed (BidStreamer) | – |

### 2.3. Форматы запросов и ответов

Тело запроса POST /lots/{lotId}/bids описывается JSON-схемой:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "PlaceBidRequest",
  "type": "object",
  "required": ["amount", "expectedPrice"],
  "properties": {
    "amount":        { "type": "number", "exclusiveMinimum": 0,
                       "multipleOf": 0.01 },
    "expectedPrice": { "type": "number",
                       "description": "цена, которую видел участник" }
  },
  "additionalProperties": false
}
```

Поле expectedPrice позволяет BidTracker понять, что участник ставил, не зная о более новой ставке, и отклонить её с кодом BID_TOO_LOW вместо ставки на неожиданную сумму. Ответ 202 Accepted подтверждает только приём ставки и содержит её идентификатор и серверную отметку времени:

```
{
  "bidId": 918273,
  "lotId": 7001,
  "amount": 1300.00,
  "status": "PENDING",
  "serverTs": "2026-10-20T18:42:35.104512Z"
}
```

После проверки BidTracker публикует результат, и BidStreamer рассылает его по WebSocket. Событие bid.accepted и ответ GET /lots/{lotId}/state используют одну и ту же структуру состояния лота:

```
{
  "lotId": 7001,
  "status": "OPEN",
  "currentPrice": 1300.00,
  "bidStep": 50.00,
  "leader": { "participantNo": 112, "isYou": false },
  "bidsCount": 17,
  "closesAt": "2026-10-20T18:42:45Z",
  "version": 17
}
```

Если ставка не прошла проверку, автору ставки отправляется событие bid.rejected с кодом причины и актуальной ценой:

```
{
  "event": "bid.rejected",
  "bidId": 918274,
  "lotId": 7001,
  "code": "BID_TOO_LOW",
  "detail": "Текущая цена 1350.00 BYN, минимальная ставка 1400.00 BYN",
  "currentPrice": 1350.00,
  "version": 18
}
```

Ошибки, обнаруженные синхронно при приёме запроса, возвращаются в формате application/problem+json (RFC 9457) с полями type, title, status, code и detail.

### 2.4. Коды ошибок

Таблица 3 – Коды ошибок API

| Где возвращается | Код ошибки | Причина |
|---|---|---|
| HTTP 400 | VALIDATION_ERROR | Тело запроса не соответствует схеме |

| Где возвращается | Код ошибки | Причина |
|---|---|---|
| HTTP 401 | UNAUTHORIZED | Токен отсутствует или истёк |
| HTTP 404 | LOT_NOT_FOUND | Лот не существует |
| HTTP 429 | TOO_MANY_REQUESTS | Превышена частота запросов (заголовок Retry-After) |
| HTTP 503 | SERVICE_UNAVAILABLE | Сервис приёма ставок временно недоступен |
| событие bid.rejected | NOT_REGISTERED | Пользователь не допущен к аукциону |
| событие bid.rejected | SELLER_CANNOT_BID | Продавец пытается ставить на свой лот |
| событие bid.rejected | BID_TOO_LOW | Пока участник ставил, цена выросла (expectedPrice устарел) |
| событие bid.rejected | LOT_CLOSED | Лот уже закрыт или ещё не открыт |
| событие bid.rejected | NOT_MULTIPLE_OF_STEP | Сумма не кратна шагу ставки |
| событие bid.rejected | LIMIT_EXCEEDED | Сумма превышает доступный лимит участника |

# РАЗДЕЛ 3. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ

## 3.1. Выбор паттерна

Для подсистемы торгов выбран архитектурный паттерн **CQRS** (Command Query Responsibility Segregation – разделение ответственности команд и запросов). Суть паттерна в том, что операции, изменяющие состояние (команды), и операции чтения (запросы) обрабатываются разными моделями и компонентами, которые синхронизируются через события.

Выбор обусловлен особенностями задачи:

- **повышение масштабируемости** – нагрузка резко несимметрична: на одну ставку приходятся тысячи чтений, так как каждое обновление получают все участники аукциона (до 5 000 по ТЗ); сторона чтения масштабируется отдельно, не нагружая обработку ставок;
- **разные требования к записи и чтению** – запись требует строгого порядка и согласованности (одна ставка за раз, серверная отметка времени), а чтение – минимальной задержки; в одной модели эти требования конфликтуют;
- **улучшение тестируемости** – логика проверки ставки сосредоточена в BidTracker и тестируется отдельно от проекций и представлений;
- **упрощение разработки** – стороны команд и запросов развиваются независимо и могут разрабатываться разными участниками фича-команды Bidding параллельно.

### 3.2. Применение паттерна в HammerLive

Применение CQRS показано на диаграмме компонентов (рисунок 3).

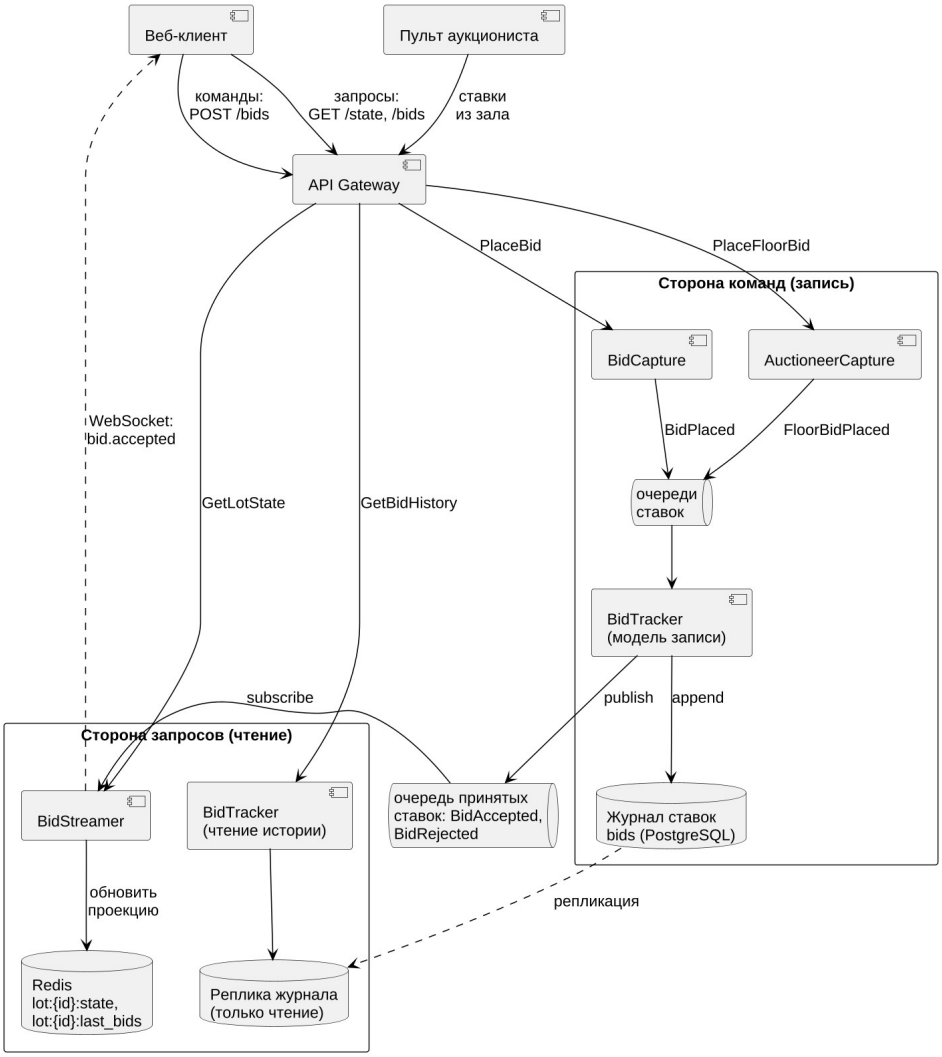



Рисунок 3 – Применение паттерна CQRS в подсистеме торгов

На стороне команд онлайн-ставки принимает BidCapture, а ставки из зала – AuctioneerCapture. Оба сервиса только фиксируют ставку и помещают её в очередь, не выполняя бизнес-проверок. Модель записи – BidTracker: он объединяет оба потока, проверяет ставку по правилам лота и списку допущенных участников, добавляет её в журнал bids (только добавление, без изменения прежних записей) и публикует результат – BidAccepted или BidRejected – в очередь принятых ставок.

На стороне запросов BidStreamer подписан на эту очередь и обновляет проекции в Redis: состояние лота lot:{id}:state и последние ставки lot:{id}:last_bids. Из них он отвечает на запрос GetLotState и рассылает события участникам по WebSocket. Полная история ставок (GetBidHistory) читается из реплики журнала BidTracker, поэтому тяжёлые запросы истории не нагружают основную базу, в которую записываются ставки.

Недостатком CQRS является итоговая (а не мгновенная) согласованность: проекции отстают от модели записи на время доставки события через очередь (десятки миллисекунд), а участник узнаёт результат

своей ставки не в ответе на команду, а событием. Поле version в состоянии лота позволяет клиенту отбрасывать устаревшие обновления.

## ЗАКЛЮЧЕНИЕ

Архитектура и API HammerLive спроектированы согласованно и прямо влияют друг на друга. Разделение команд и запросов по паттерну CQRS отражено в самом API: команды выполняются методами POST, PUT и DELETE и попадают через BidCapture на сторону записи, запросы GET обслуживаются BidStreamer из проекций стороны чтения, а результаты работы модели записи доставляются клиенту через канал WebSocket. Благодаря этому API Gateway может направлять запросы чтения на горизонтально масштабируемый сервис, не нагружая обработку ставок.

В обратную сторону архитектура определила требования к API. Асинхронная обработка ставок через очереди определила ответ 202 Accepted на команду и разделение ошибок на синхронные (HTTP) и бизнес-отказы, приходящие событием bid.rejected; возможные повторы потребовали заголовка Idempotency-Key. Строгий порядок ставок в модели записи породил код BID_TOO_LOW и поле expectedPrice, а итоговая согласованность проекций – поле version в состоянии лота. Модель данных служит основой обеих сторон: журнал bids в базе BidTracker является источником истины, а проекции BidStreamer в Redis строятся из событий о его изменениях. Таким образом, решения на уровне данных, API и архитектуры образуют единую систему, отвечающую требованиям ТЗ к скорости, порядку обработки ставок и масштабируемости.